Statistics and Data Science

LEPL1109: Data science part

L. Jacques

– Part I –

Supervised Learning

Part 1: Supervised Learning

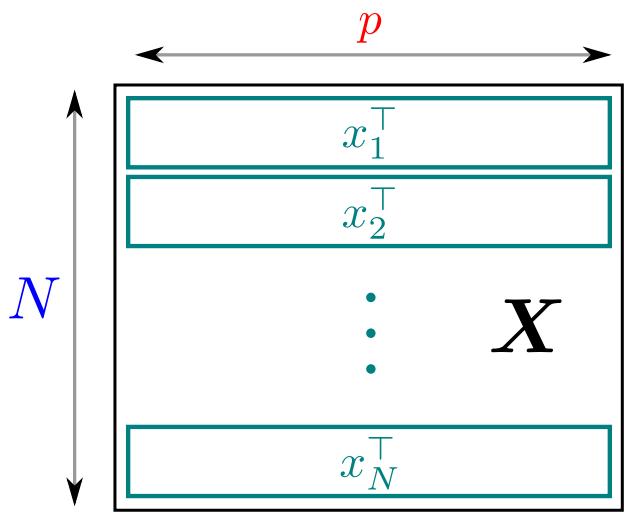
1.1. Introduction to SL

1.2. First Steps in SL

1.3. Models for SL

Matrix, vector, and set conventions:

- $\{a_i\}_{i=1}^N \equiv \{a_1, \ldots, a_N\} \equiv \{a_i : 1 \leqslant i \leqslant N\}$ (a set of indexed elements)
- We'll often consider sets of (input) vectors in p-dimensional spaces, *e.g.*, $\{x_i\}_{i=1}^N$, with $x_i \in \mathbb{R}^p$ (*i.e.*, $\{x_i\}_{i=1}^N \subset \mathbb{R}^p$).
- The dimension "N" is reserved for the size of a (data)set.
- All matrices (and vectors) with N rows (resp. components) are **bold**, *e.g.*, $\boldsymbol{X} \in \mathbb{R}^{N \times p}$, $\boldsymbol{u} = (u_1, \cdots, u_N)^\top \in \mathbb{R}^N$.



$$\equiv \{x_i\}_{i=1}^N \subset \mathbb{R}^p$$

Stochastic quantities and notations:

- Random variables or random vectors (*r.v.*) denoted by X, Y, ...
- We will often write:

  $y \equiv$ sample of the *r.v.* Y

  $x = (x_1, \cdots, x_p)^\top \equiv$ sample of the *r.v.* $(X_1, \cdots, X_p)^\top$.

As usual, we will write:

- Probability $\equiv P$ (sometimes $\mathbb{P}$)
- Expectation $\equiv \mathbb{E}$
- Variance $\equiv \mathbb{V}$ or Var.

Example: If X represents a fair coin, with 'H'ead/'T'ail (or ± 1) (aka Bernoulli)

- $P(X = \texttt{H} = 1) = \mathrm{p}_X(1) = \frac{1}{2}$, and $P(X = \texttt{T} = -1) = \mathrm{p}_X(-1) = \frac{1}{2}$,
- $\mathbb{E}\,X := \sum_{k \in \{\pm 1\}} k\ \mathrm{p}_X(k) = P(X = 1) - P(X = -1) = 0$,
- $\mathbb{V}X := \sum_{k \in \{\pm 1\}} (k - 0)^2\ \mathrm{p}_X(k) = P(X = 1) + P(X = -1) = 1$.

Joint discrete/continuous formalism:

- The pmf/pdf of a discrete/continuous *r.v.* X is "p_X".
- This means that, for instance:

$$P(X \in I) = \sum_{x \in I} \mathrm{p}_X(x) \text{ (if pmf)}, \quad P(X \in I) = \int_{x \in I} \mathrm{p}_X(x)\,\mathrm{d}x \text{ (if pdf)}$$

with $\mathrm{p}_X(x) = 0$ outside of X's *range* (*i.e.*, X's domain)

Examples: $P(X \geqslant 2) = \sum_{x=2}^{+\infty} \mathrm{p}_X(x)$, $P(0 \leqslant X \leqslant 1) = \int_0^1 \mathrm{p}_X(x)\,\mathrm{d}x$

Remarks: slight difference with DH's notation.

Abuse of notations: When clear from the context,

- $\mathrm{p}(U)$, or just p, denotes p_U, the pdf of the *r.v.* U; (upper-case U)
- $\mathrm{p}_U(u)$, or $\mathrm{p}(U = u)$, or just $\mathrm{p}(u)$, (lower-case u)
  denotes the evaluation of the pdf of U on u.

Let's start with a few notations and convention (4/4)

Miscellaneous: (come back to this when you'll study the slides)

- The set of the first N integers is $[N] := \{1, \cdots, N\}$;
- Sets are often denoted by "calligraphic" letters, *e.g.*, $\mathcal{C} \subset [N]$, $\mathcal{S} \subset \mathbb{R}^p$
- Indicator $I(a \neq b) = 1$ if $a \neq b$ and 0 otherwise;
- $|\mathcal{S}|$ = size (cardinality) of $\mathcal{S}$.
- Some symbols and abbreviations used in these slides: "$\equiv$" $\rightarrow$ "*equivalent to*", ":=" $\rightarrow$ "*defined as*", "i.i.d." $\rightarrow$ "*identically and independently distributed*", "s.t." $\rightarrow$ "*such that*" "*wrt*" $\rightarrow$ "*with respect to*", "*i.e.,* " $\rightarrow$ "*that is*", "*e.g.,* " $\rightarrow$ "*for instance*", "*vs.*" $\rightarrow$ "*versus*", "aka" $\rightarrow$ "also known as", "ctd" $\rightarrow$ "continued", "ONB" $\rightarrow$ "orthonormal basis", "wlog" $\rightarrow$ "*without loss of generality*", "cst" $\rightarrow$ "constant".
- The slides contains *hyperlinks* in magenta, such as LEPL1109, either to external references, or to internal ones (pages).

Supervised Learning (SL): Regression vs Classification (1/3)

Most common **supervised learning** tasks: *Regression* and *Classification*

$\Rightarrow$ Both are doing **prediction**, but on different objects.

1. Regression: prediction of ordered *values* (or quantitative values)

Examples:

- "What will be the patient *temperature* in 2h?" (continuous value)
- "How many people will be infected by covid-19 tomorrow?" (integer value)

2. Classification: prediction of classes or categories (or qualitative values)

Examples:

- "What is the *disease* of a patient given some symptoms?" (category)
- "Which film category to recommend from a watching list?" (category)

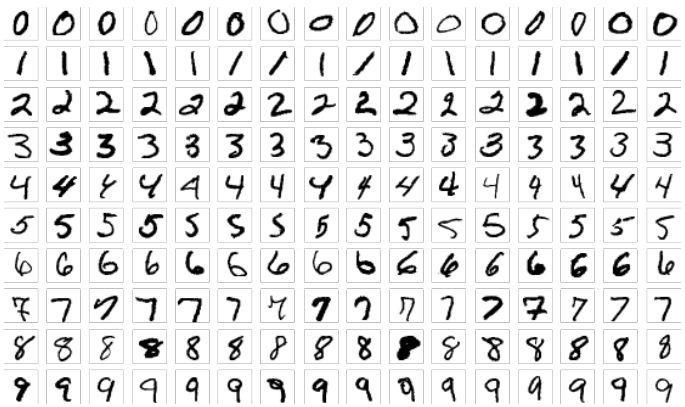
Questions addressed in this course

- How to learn to predict from available data? Prediction quality?
- How to correctly use the available data? (*e.g.*, using cross-validation)

Regression or Classification? Distinction not always easy.

Examples:

- Predict intensity of secondary effects to a tested drug
  if *intensity* $\in$ {"none", "moderate", "severe"}
- Predict the *value* (*i.e.*, in {"0", ..., "9"}) of a handwritten digit
  from a 28 $\times$ 28 image (aka MNIST problem) ?



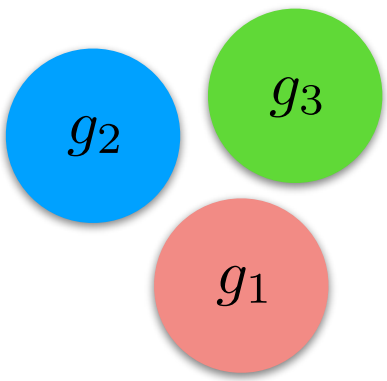
For binary 0/1 data, no distinction between regression and classification.

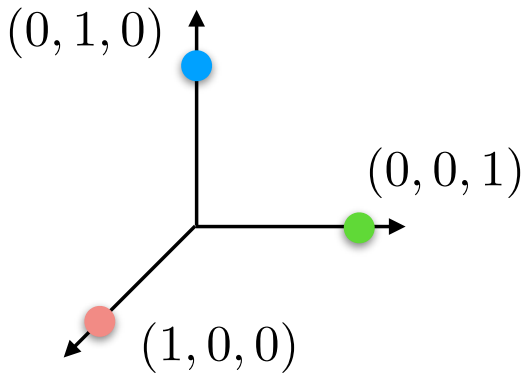
Turning classification into a regression?

Dummy variable (aka "*one-hot encoding*"):

Transform *qualitative values* $\rightarrow$ *quantitative values* (in high dimension)

$$G \in \mathcal{G} = \{g_1, \dots, g_K\} \qquad Y \in \mathbb{R}^K$$





$$G = g_k \qquad Y = e_k := (0, \cdots, 0, \underset{\uparrow}{1}, 0, \cdots, 0)^\top$$

k^{th} component

Supervised Learning (SL): Linking an input to an output

Supervised learning scenario: (other names ↓)

- Input $X = (X_1, \cdots, X_p)^\top \in \mathbb{R}^p$ $\equiv$ feature/predictors/covariates
- Output $Y \in \mathcal{Y}$ (quantitative or qualitative) $\equiv$ outcome/label/response

Assumption: Y and X are (imperfectly) related (with a bit of "noise")

$\Rightarrow$ There exists a function $f : \mathbb{R}^p \to \mathcal{Y}$ such that:

(i) *for regression* ($\mathcal{Y} = \mathbb{R}$):

$$Y = f(X) + \underbrace{\varepsilon}_{\text{uncertainty}},$$

for some random noise $\varepsilon \in \mathbb{R}$ (*e.g.*, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$),

(ii) *for binary classification* (*e.g.*, $\mathcal{Y} = \mathcal{G} := \{0, 1\}$):

$$\left.\begin{aligned} P(Y = f(X)) &= 1 - \eta \\ P(Y \neq f(X)) &= \eta, \end{aligned}\right\} \text{uncertainty}$$

for some misclassification error $0 \leqslant \eta \leqslant 1$.

Supervised Learning (SL): Objective

In theory: (statistical decision theory, SL3)

> Find an estimate $\hat{f}$ of f for doing predictions (or inferences) of Y given X.

In practice:

> Find a reliable estimate $\hat{f}$ of f for doing predictions (or inferences) of a sampling of Y given a sampling of X (aka the dataset).

Remark: the function f

- could be unknown, or very involved,
- *or . . . we just don't care of f.*

Question: How to find the function $\hat{f}$?

Supervised Learning (SL): Training/Testing

A. Training stage:

1. Get a "representative" data set $\mathcal{T}$ of your task (ideally large), *i.e.*, a sampling of (X, Y), a set of N measured pairs of {"*input*", "*label*"}:

$$\mathcal{T} := \{(x_i, y_i)\}_{i=1}^{N} \subset \mathbb{R}^p \times \mathbb{R}, \quad \text{or} \quad \subset \mathbb{R}^p \times \mathcal{G}.$$

2. Clean/preprocess/transform the data, *e.g.*,
   - Imputing missing values (*e.g.*, `NAN`),
   - Encoding categorical features (*e.g.*, using dummy vars),
   - Removing outliers / Normalizing or standardizing numerical features,
   - Dimensionality reduction, ...
3. Train/fit a machine learning model (find the model parameters)

B. Prediction/Testing stage:

1. Clean/preprocess/transform new/independent data (⚠ exactly as in #2)
2. Apply the model on them to make predictions.

Key question💰: Evaluate the learned-model performance without new data?
⇒ *Solution*: resampled your dataset, cross-validation, ... (more on this later)

Example 1: Regression (from ISL book)

- $X_1, \cdots, X_p$ are characteristics of patients' blood sample, example:

Characteristic	Unit	Value (X)
Hemoglobin	g/dl	11.0±1.0
Platelet	xx$10^{(5/)}\mu$l	3.21±0.90
Protein	g/dl	7.05±0.51
Albumin	g/dl	4.11±0.34
BUN	mg/dl	17.5±5.4
Glucose	mg/dl	45.5±20.0
Cholesterol	mg/dl	111±30
Tryglyceride	mg/dl	45.4±45.3
Phosphate	mg/dl	4.52±1.47
Calcium	mg/dl	9.21±0.56
GOT	IU/l	53.7±19.0
GPT	IU/l	74.6±57.7
CRP	mg/dl	0.111±0.219

- Y encodes patient's risk level for severe adverse reaction to a specific drug

Question: Predict Y using X to avoid giving this drug if bad for patients.

$\rightarrow$ Better to well estimate the prediction error beforehand.

Example 2: Classification, dataset of handwritten digits (or MNIST)

- each $x_i \in \mathbb{R}^{p=28^2}$ is a 28×28 pixel image,
- and $y_i \in \mathcal{G} := \{\text{'0'}, \cdots, \text{'9'}\}$ a *label*.

For instance, for $i = 36\,000$ over the $N = 70\,000$ MNIST images,

$$x_i = \boxed{} \in \mathbb{R}^{28\times 28} \simeq \mathbb{R}^{784},\ y_i = \text{'5'}$$

and the dataset $\mathcal{T}$ looks like



How to predict the number (in $\mathcal{G}$) written on a new image x ($\notin \mathcal{T}$)?

$\Rightarrow$ The relation between X and $Y \in \{\text{'0'}, \cdots, \text{'9'}\}$ can be very involved.

1.1. Introduction to SL

1.2. First Steps in SL

1.3. Models for SL

What are we going to see now ...

First steps in SL in the context of classification.

We analyze 2 simple methods to classify data:

1. a binarized linear model,
   i.e., using least squares + thresholding
2. the k-nearest neighbors method (k-NN).

Many modern ML techniques are based on LS or k-NN.

The next slides are going to be quite exploratory ...

Multivariate Linear Modeling: definition

Given a vector of inputs (or predictors) $X = (X_1, \cdots, X_p)^\top$,
predict the (scalar) output Y *assuming* the linear model

$$\hat{Y} = f_{\hat{\beta}}(X) := \sum_{j=1}^{p} X_j \hat{\beta}_j = (X_1, \ldots, X_p) \begin{pmatrix} \hat{\beta}_1 \\ \vdots \\ \hat{\beta}_p \end{pmatrix} = X^\top \hat{\beta}, \tag{1}$$

with the coefficient vector $\hat{\beta} = (\hat{\beta}_1, \cdots, \hat{\beta}_p)^\top$

This model is:

- linear, *i.e.*, $f_\beta(aX + bX') = af_\beta(X) + bf_\beta(X')$, $\forall X, X' \in \mathbb{R}^p, \forall a, b \in \mathbb{R}$, $\forall \beta$;
- a global, parametric model, with p *parameters* (stored in $\hat{\beta} \in \mathbb{R}^p$);
- bias free, $\hat{\beta}_0 = 0$.

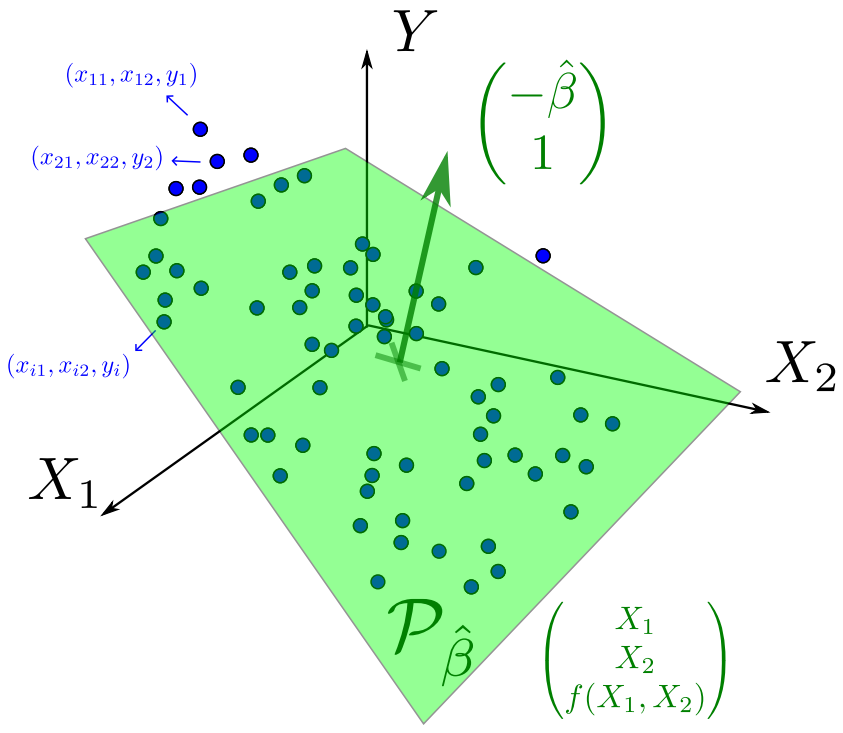
Remark: Easy to turn (1) into

$$f_{\hat{\beta}}(X) := \hat{\beta}_0 + \sum_{j=1}^{p} X_j \hat{\beta}_j$$

where $\beta_0 \in \mathbb{R}$ is a non-zero bias/intercept.

Multivariate Linear Modeling: geometrical interpretation

Geometrically: Given a dataset $\{(x_i \in \mathbb{R}^p, y_i \in \mathbb{R})\}_{i=1}^N \subset \mathbb{R}^{p+1}$ (in blue)



we want to fit a (hyper) plane

$$\mathcal{P}_{\hat{\beta}} := \{(X, f_{\hat{\beta}}(X))^\top : \forall X \in \mathbb{R}^p\} = \{(X, X^\top \hat{\beta})^\top : \forall X \in \mathbb{R}^p\} \subset \mathbb{R}^{p+1}$$

through the data $\Leftrightarrow$ finding an optimal $\hat{\beta}$.

Remark: $0 \in \mathcal{P}_{\hat{\beta}}$ and the normal to $\mathcal{P}_{\hat{\beta}}$ is $\begin{pmatrix} -\hat{\beta} \\ 1 \end{pmatrix}$.

How to fit this model (*i.e.*, find an optimal $\hat{\beta}$) to a set of training data?

Given the *training set*

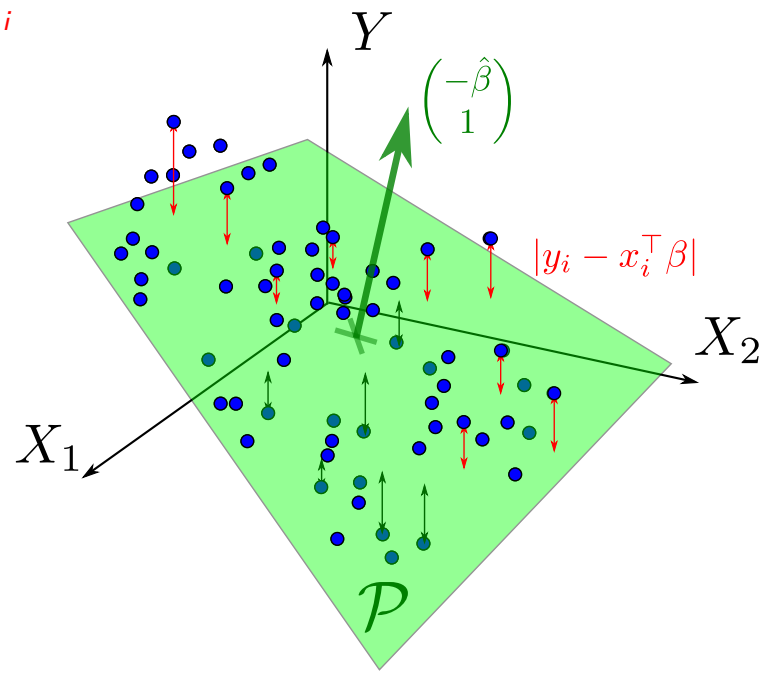
$$\mathcal{T} := \{(x_i, y_i)\}_{i=1}^N \subset \mathbb{R}^p \times \mathbb{R}$$

we pick $\hat{\beta}$ as the minimizer of the empirical risk (with the *squared loss*)

$$\hat{R}(\beta) := \frac{1}{N} \sum_{i=1}^N (y_i - \underbrace{x_i^\top \beta}_{=(\boldsymbol{X}\beta)_i})^2 = \frac{1}{N} \|\boldsymbol{y} - \boldsymbol{X}\beta\|_2^2.$$

with

- $\boldsymbol{X} = (x_1, \cdots, x_N)^\top \in \mathbb{R}^{N \times p}$
- $\boldsymbol{y} = (y_1, \cdots, y_N)^\top$, $\|\boldsymbol{u}\|_2^2 := \boldsymbol{u}^\top \boldsymbol{u}$.



Remarks:

- we minimize the sum of squared vertical distances;
- ⚠ **but not** the "distance" between $\mathcal{T}$ and $\mathcal{P}$,

  i.e., LS $\neq$ PCA (see UL section).

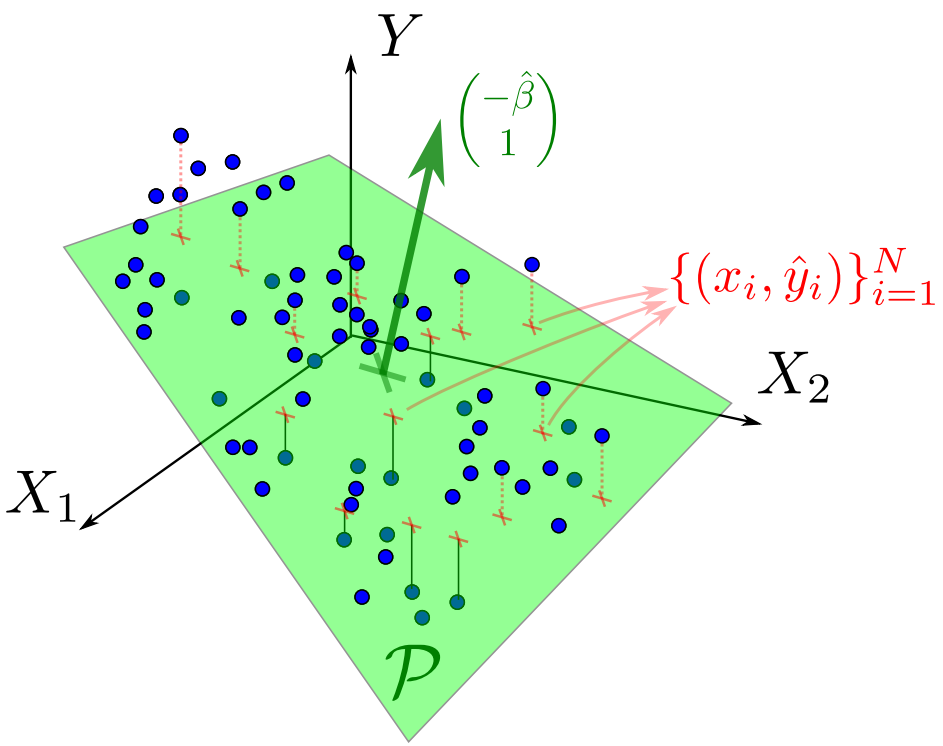
Solution: $\hat{R}$ minimum on $\hat{\beta}$ if (⚠ assuming $N > p$ and $\boldsymbol{X}^\top \boldsymbol{X}$ invertible)

$$\nabla_\beta \hat{R}(\hat{\beta}) = 2\boldsymbol{X}^\top(\boldsymbol{y} - \boldsymbol{X}\hat{\beta}) = 0 \;\Rightarrow\; \hat{\beta} := \boldsymbol{X}^\dagger \boldsymbol{y} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top \boldsymbol{y}.$$

Fitting: Using the model, *i.e.*, $\hat{\beta}$, the vector

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\beta} = (x_1^\top \hat{\beta}, \cdots, x_N^\top \hat{\beta})^\top = \boldsymbol{H}\boldsymbol{y} = \boldsymbol{X}\boldsymbol{X}^\dagger \boldsymbol{y}$$

$\Rightarrow$ N values of $\boldsymbol{y}$ fitted by the $N \times N$ "hat matrix" $\boldsymbol{H} := \boldsymbol{X}\boldsymbol{X}^\dagger$.



Simple artificial 2-D classification:

$$\mathcal{T} = \{(x_i \in \mathbb{R}^2, y_i \in \{0, 1\})\}_{i=1}^{N=200}.$$

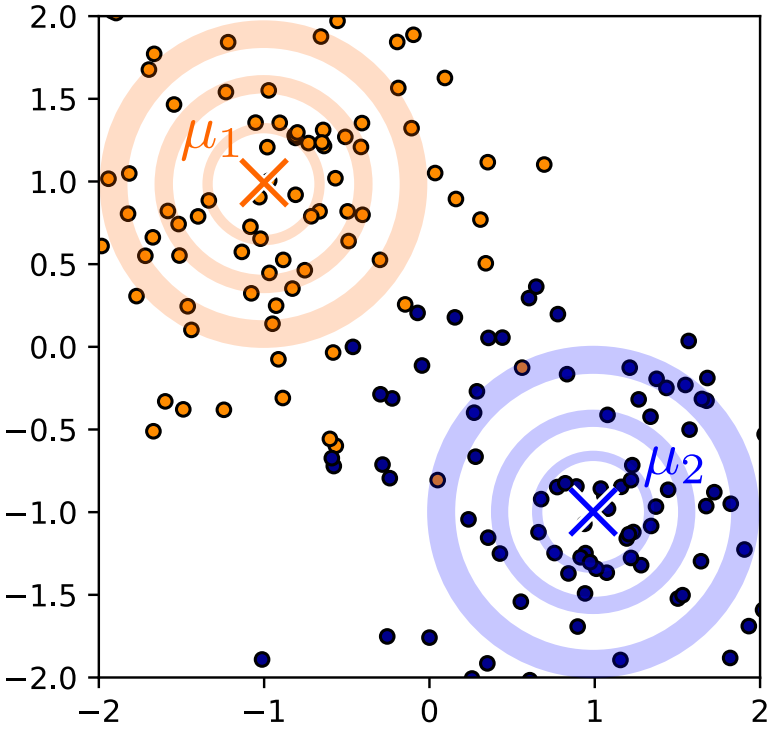
with:

- 1st class (label $y_i = 1$, ●): $x_i \sim \mathcal{N}(\mu_1, \sigma^2 \boldsymbol{I})$
- 2nd class (label $y_i = 0$, ●): $x_i \sim \mathcal{N}(\mu_2, \sigma^2 \boldsymbol{I})$
- $\mu_1 := (-1, 1)^\top$ and $\mu_2 := (1, -1)^\top$
- $\sigma = 0.8$, 100 points per class.

Reminder:

$$\mathcal{N}(\mu, \sigma^2 \boldsymbol{I}) \sim c_\sigma \exp(-\frac{\|x-\mu\|^2}{2\sigma^2}),$$

with $x, \mu \in \mathbb{R}^2$, $\sigma > 0$, $c_\sigma := (2\pi\sigma^2)^{-p/2}$.




```
from sklearn.datasets \
     import make_blobs

centers = [[-1, 1], [1, -1]]
X, y = make_blobs(n_samples=200,
                  centers=centers,
                  cluster_std=0.8)
```

Training LS on $\mathcal{T}$, we get $\hat{\beta}$.

LS regression provides a continuous $\hat{Y}$:

$$\hat{Y}(X) = f_{\hat{\beta}}(X) = X^\top \hat{\beta} \in \mathbb{R}.$$

However, $Y \in \{0, 1\}$ in our example.

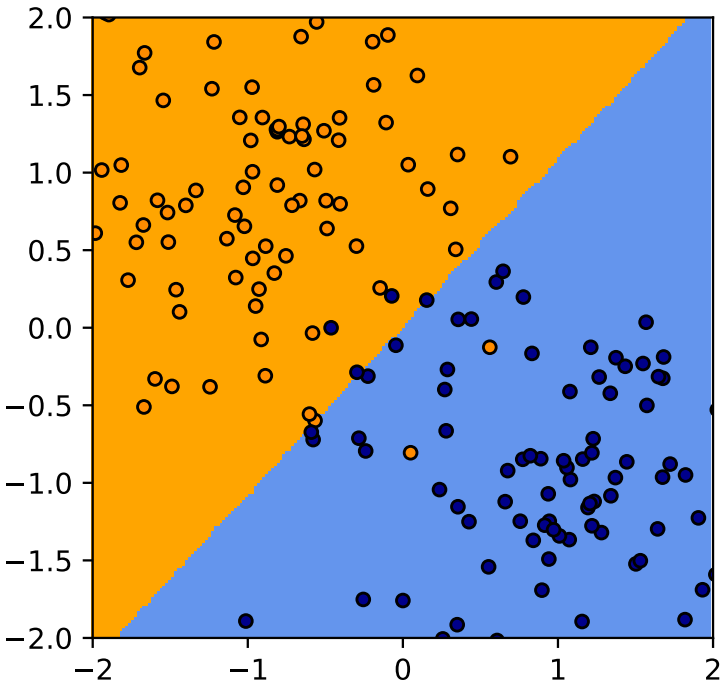
LS classifier = LS regression
+ binary conversion $\mathcal{B}_\tau$

Given a *threshold* $\tau > 0$ and $x \in \mathbb{R}^2$,
the estimated class $\hat{G}$ of x is

$$\hat{G}(x) = \mathcal{B}_\tau(x^\top \hat{\beta}) := \begin{cases} 1 & \text{if } (x^\top \hat{\beta}) > \tau, \\ 0 & \text{if } (x^\top \hat{\beta}) \leqslant \tau, \end{cases}$$

Notes:

- other values for τ are possible (when ❓)
- $-\tau$ can be set to the *intercept*.



Threshold set to $\tau = 0.5$.

```
from sklearn import linear_model
LS = linear_model.LinearRegression()
LS.fit(X,y)

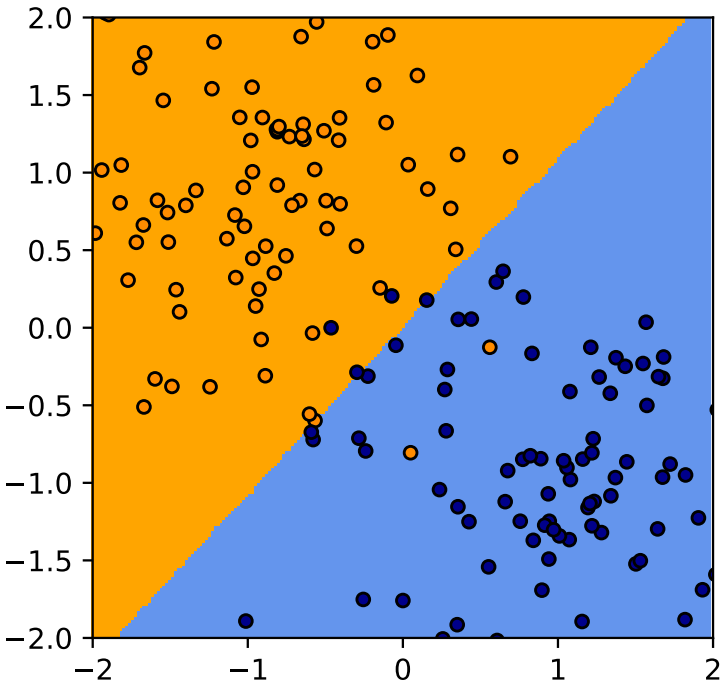
# (then use LS.predict()
#       + "binary conversion"
#          on a 2-D domain)
```

Conclusions:

- Geometric interpretation is there!

  Orange area: $\{x : x^\top \hat{\beta} > 0.5\}$
  Blue area: $\{x : x^\top \hat{\beta} < 0.5\}$
- Linear boundary: $\{x : x^\top \hat{\beta} = 0.5\}$
- Only a few misclassification errors on training samples.

Is it too easy?



Threshold set to $\tau = 0.5$.

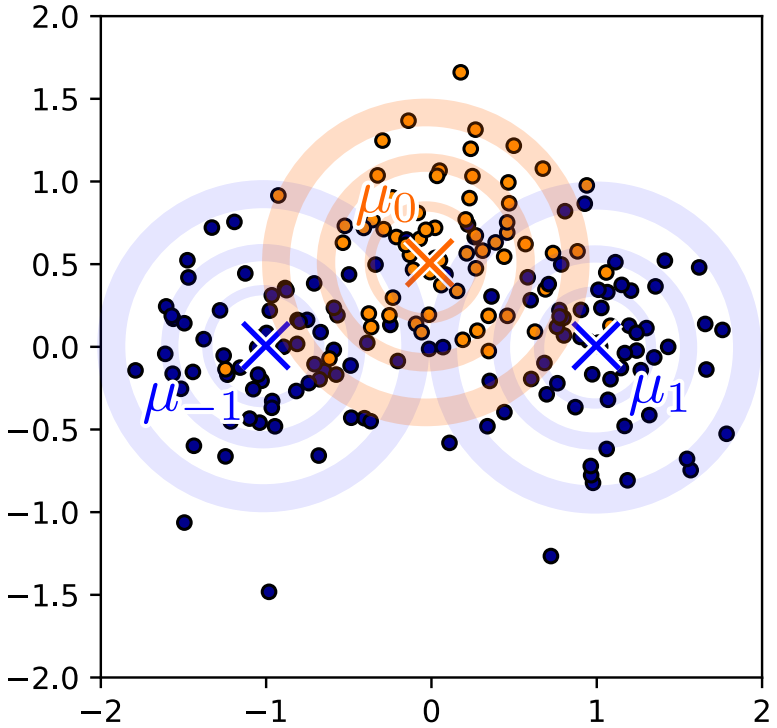
```
from sklearn import linear_model
LS = linear_model.LinearRegression()
LS.fit(X,y)

# (then use LS.predict()
#       + "binary conversion"
#          on a 2-D domain)
```

Consider this more **advanced case**.

Setup: Dataset $\mathcal{T}$ made of

- Sampling of 3 Gaussian blobs on $\mu_0 = (0, 0.5)^\top$ & $\mu_{\pm 1} := (\pm 1, 0)^\top$
- 1st class (label 1, ●): blob on μ_0
- 2nd class (label 0, ●): 2 blobs on $\mu_{\pm 1}$
- $N = 200$ ($\frac{2}{3}$ for ●, $\frac{1}{3}$ for ●), $\sigma = 0.4$

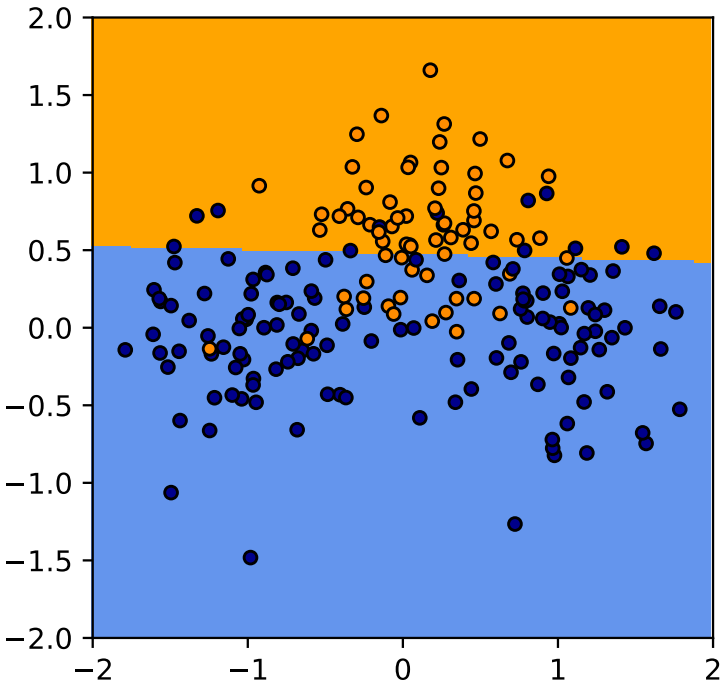


Results of binarized LS regression on $\mathcal{T}$:

- Many misclassified training samples, for the two classes.

Maybe,

- wrong assumption on the model (linearity fails here)?
- the model is too rigid?
- need of a non-linear boundary?
- or unavoidable model errors? Maybe LS cannot be optimal here?



Threshold set to $\tau = 0.5$.

Moreover, problem of interpretability! Before binarization by $\hat{G}$,

$$\text{does it make sense to have } \hat{Y} < 0?? \quad \text{or } \hat{Y} > 1??$$

while $Y \in \{0, 1\} \subset [0, 1]$. (more on this later)

k-Nearest Neighbors Method

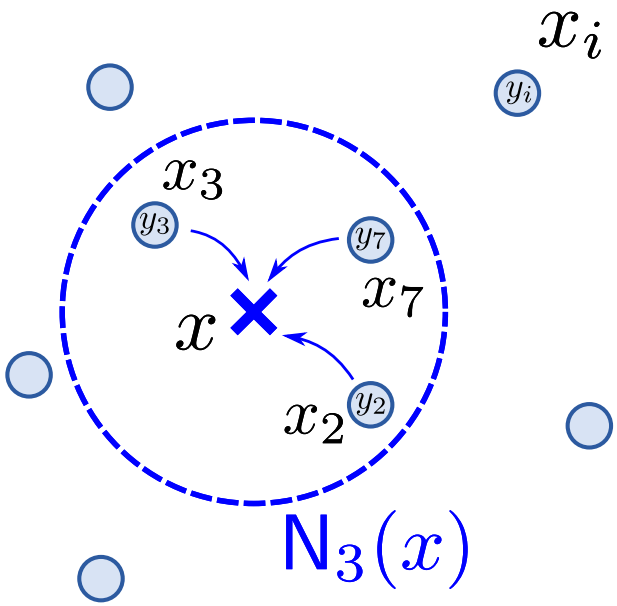
Given a number of neighbors $k \in \mathbb{N}$, we first define

$$\mathsf{N}_k(x): \text{ the set of } k \text{ closest points to } x \text{ in } \{x_i\}_{i=1}^N \subset \mathbb{R}^p.$$

In k-**Nearest Neighbors** (or k-NN) regression (or *fit*)

the value $\hat{Y}(x)$ *assigned to* x = *the average of the point's label in* $\mathsf{N}_k(x)$.

$$\hat{Y}(x) = \frac{1}{k} \sum_{i:\, x_i \in \mathsf{N}_k(x)} y_i.$$



$\Rightarrow$ k-NN is a local average, it is also non-parametric.

k-NN classifier = k-Nearest Neighbor regression + binary conversion $\mathcal{B}_\tau$

with the (same) binary conversion (at *threshold* $\tau = 0.5$):

$$\hat{G}(x) = \mathcal{B}_\tau\left(\hat{Y}(x)\right) := \begin{cases} 1 & \text{if } \hat{Y}(x) > \tau, \\ 0 & \text{if } \hat{Y}(x) \leqslant \tau, \end{cases}$$

Equivalence:

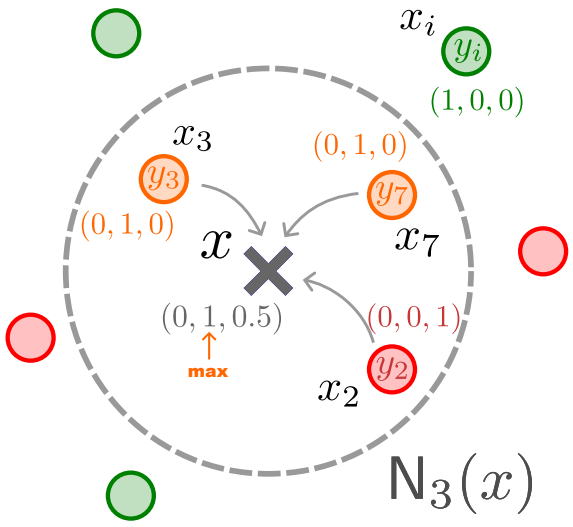
The k-NN classifier on binary labels with $\tau = 0.5 \Leftrightarrow$ "majority voting"

$$\hat{G}(x) = \begin{cases} 1 & \text{if } \left|\{x_i \in \mathsf{N}_k(x) : y_i = 1\}\right| > \left|\{x_i \in \mathsf{N}_k(x) : y_i = 0\}\right| \\ 0 & \text{otherwise,} \end{cases}$$

wit $|\mathcal{S}|$ the cardinality of a set $\mathcal{S}$.

For $K \geqslant 2$ classes: to define $\hat{G}(x)$

- apply *one-hot encoding* to all y_i's (see p. 8)
- Next apply k-NN regression
- Assign to x the index of the maximum



Equivalences:

k-NN classifier $\hat{G}(x) =$

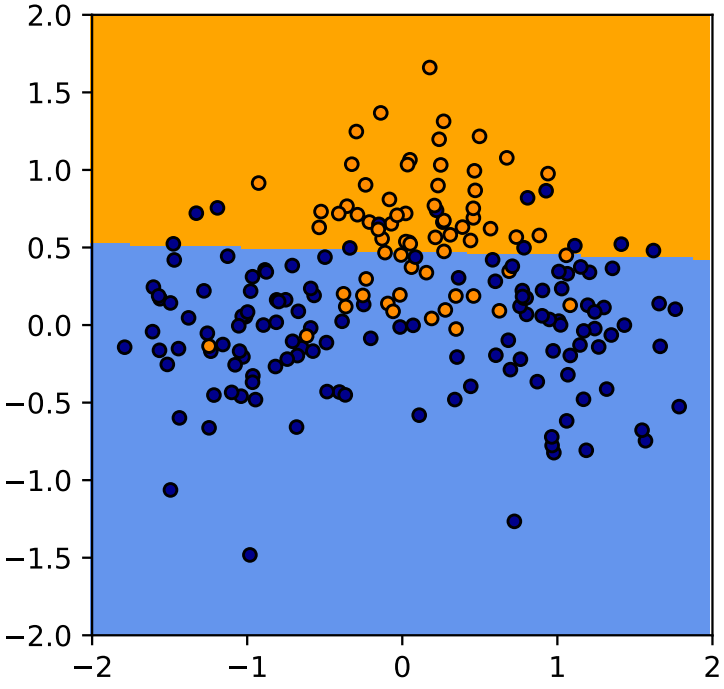
$\Leftrightarrow$ *most represented class in* $N_k(x)$

$\Leftrightarrow$ *class with highest number of votes* *(majority voting)*

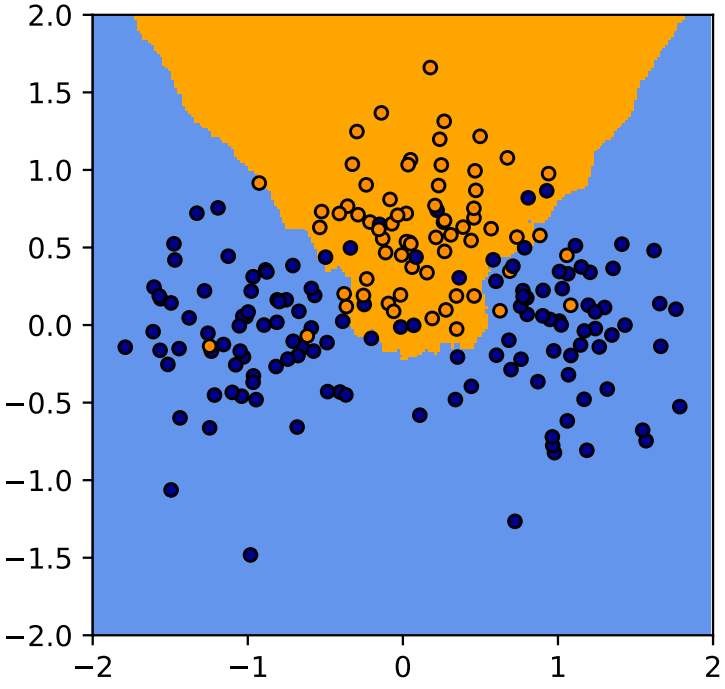
$$\hat{G}(x) = \arg\max_{g \in \mathcal{G}} \left| \{x_i \in N_k(x) : y_i = g \right| \quad \in \mathcal{G},$$

with $\mathcal{G} = \{g_1, \ldots, g_K\}$.

Previous LS classifier



k-NN classifier with $k = 10$



Threshold set to $\tau = 0.5$.

```
from sklearn import neighbors
kNN = neighbors.KNeighborsClassifier(
                n_neighbors = 10
    )
kNN.fit(X,y)

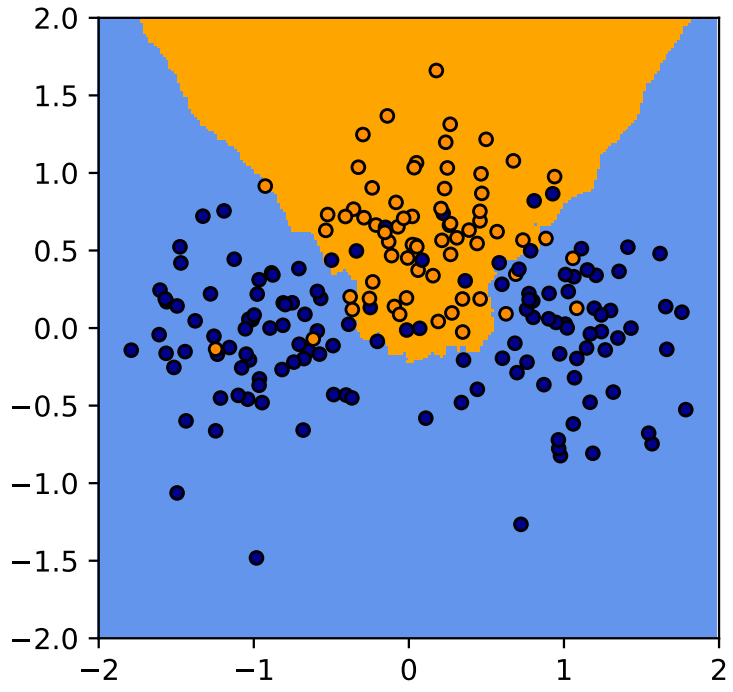
# (then use kNN.predict() on a 2-D domain)
# (note: "binary conversion" included)
```

Conclusions:

- in orange area, more ● neighbors around x than ● neighbors.
- lower classification errors than for LS.
- irregular decision boundaries ($\neq$ for LS).
- boundaries shaped by local clusters where one class dominates.

What happens if we change k?

k-NN classifier with $k = 10$



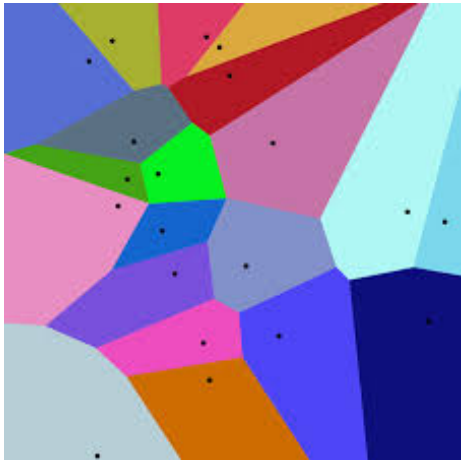
Threshold set to $\tau = 0.5$.

```
from sklearn import neighbors
kNN = neighbors.KNeighborsClassifier(
                n_neighbors = 10
    )
kNN.fit(X,y)

# (then use kNN.predict() on a 2-D domain)
# (note: "binary conversion" included)
```

For $k = 1$:

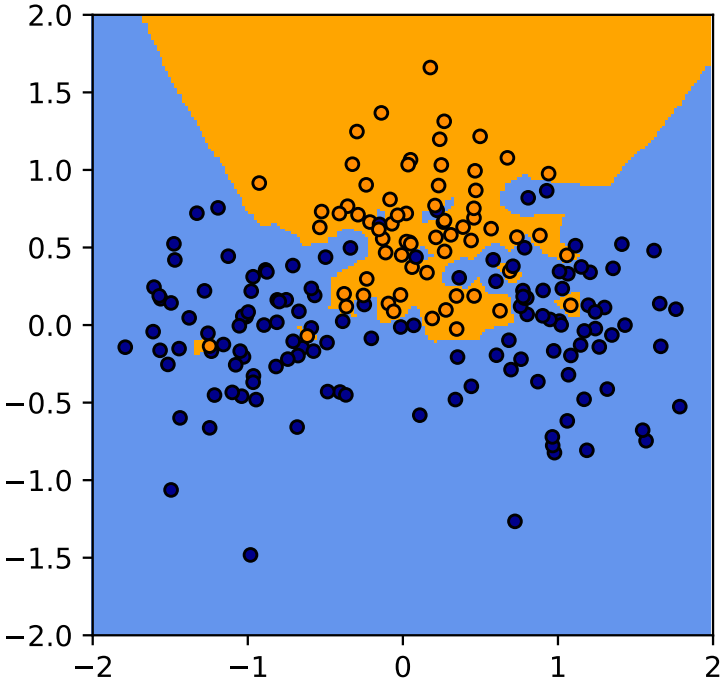
- Classification regions
  = Voronoi tessellation.



- All training samples are correctly classified
  $\Rightarrow$ no *training error*

Is it good?

k-NN classifier with $k = 1$



Threshold set to $\tau = 0.5$.

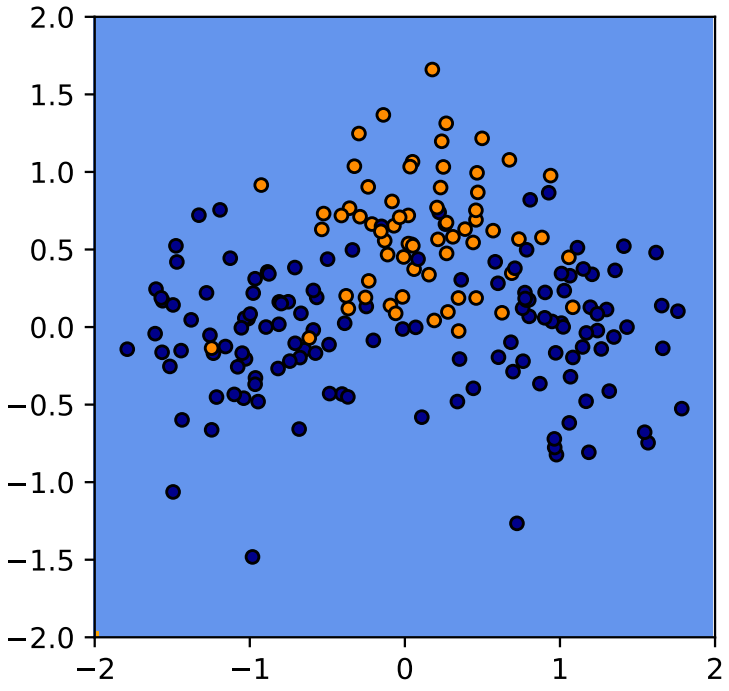
```
from sklearn import neighbors
kNN = neighbors.KNeighborsClassifier(
                n_neighbors = 1
    )
kNN.fit(X,y)

# (then use kNN.predict() on a 2-D domain)
# (note: "binary conversion" included)
```

For k too large:

- Classification regions = ?? What happened here?
- Global majority voting: the largest class (the blue) wins all votes.
- Larger *training error* than for $k = 1$ (with no training error).

k-NN classifier with $k = 200$



Threshold set to $\tau = 0.5$.

```
from sklearn import neighbors
kNN = neighbors.KNeighborsClassifier(
                n_neighbors = 200
    )
kNN.fit(X,y)

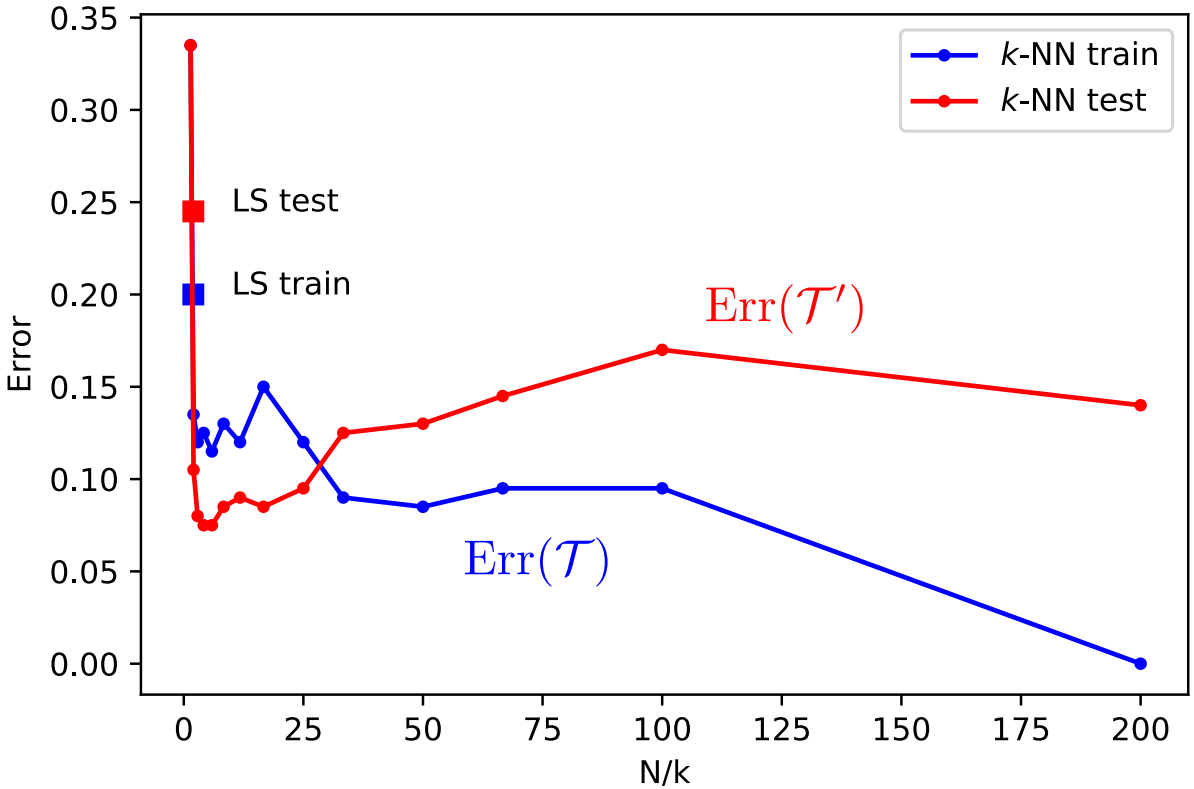
# (then use kNN.predict() on a 2-D domain)
# (note: "binary conversion" included)
```

k-Nearest Neighbors: trade-off between small/large k

Classification error of a classifier $\hat{f}$ on a given dataset $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^N$:

$$\text{Error} := \text{Err}(\mathcal{T}) = \frac{1}{N} \sum_{i=1}^N I\left(y_i \neq \hat{f}(x_i)\right).$$

Is k-NN a good classifier if $N/k \uparrow$? Not really



Training error of LS and k-NN classifiers.

Training on $\mathcal{T}$, testing on $\mathcal{T}' = \{(x'_j, y'_j) \sim (X, Y)\}_{j=1}^N \neq \mathcal{T}$.

k-Nearest Neighbors: conclusion

In summary,

Pros of k-NN:

- No *explicit* model assumption (X, Y) $(\neq$ LS regression)

  ... ⚠ we'll see later there is one *implicit* behavior imposed on $\hat{Y}$.
- Non-linear, irregular boundary
- Controlled misclassification error on training samples, or *training error*
  $\Rightarrow$ if $k \downarrow$, that training (misclassification) error $\downarrow$

Cons:

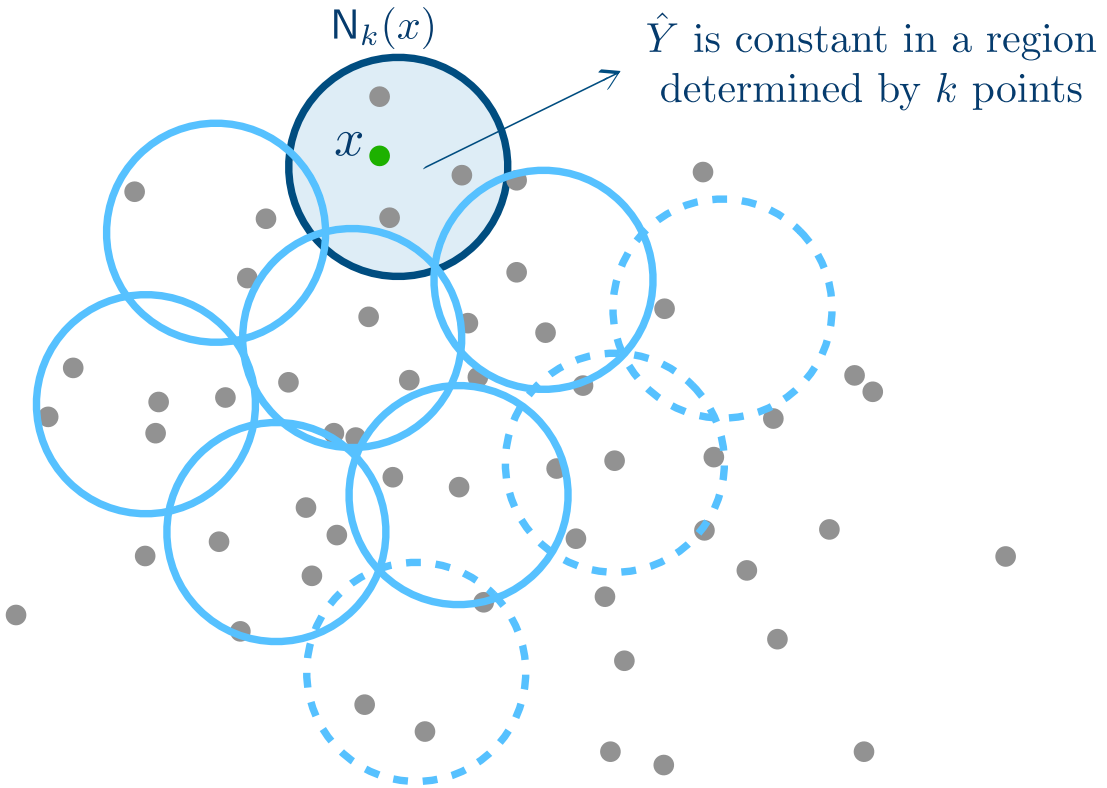
- Training error can be small, but this does not guarantee a good classifier
  $\Rightarrow$ What is the rrror on a new independent dataset, or *test* set?
- Hyper-parameter k to adjust $\Rightarrow$ *model selection*
- k-NN depends on the type of distance ($\rightarrow$ data preprocessing) (see TP)

Effective number of parameters for k-NN

Question: What is the *effective* number of (free) parameters in k-NN?

(in the same sense that $f_\beta(x) := x^\top \beta$ has p parameters if $x, \beta \in \mathbb{R}^p$)

The effective number of parameters (or *flexibility*) of k-NN $= \boxed{\frac{N}{k}}$ (⚠ not k)



N/k neighborhoods $\approx$ # of "free" parameters of $\hat{Y}$

LS vs. k-NN

In summary

	Assumptions	Accuracy	Stability	# eff. param.	Comp. Time
LS	Strong (linear)	low	high	p	Large
k-**NN**	Mild	low/high	high/low	$\approx N/k$	Low

Remark: *stability* $\equiv$ variability *wrt* training dataset (more on this later)

Additional remarks:

- LS can be proved to be optimal for 2 classes generated by Gaussians.
- For more than 2 classes, a linear decision is not optimal.

1.1. Introduction to SL

1.2. First Steps in SL

1.3. Models for SL

Supervised learning objective:

Using an algorithm $\mathcal{A}$ to learn a prediction function $\hat{f}$
from a dataset $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{N}$.

This means that, $\hat{Y} = \hat{f}(X)$ predicts $Y = f(X)$ if $\mathcal{T}$ *samples well* (X, Y).

Problem: Impossibility to solve this problem without assumptions (models).

$\Rightarrow$ we assume that $f \in \mathcal{F}$, with $\mathcal{F}$ a restricted class of functions (or *prior*)

Simplified objective: $\mathcal{A}$ can only pick functions in $\mathcal{F}$:

$$\mathcal{A} : (\mathcal{T}, \mathcal{F}) \quad \rightarrow \quad \hat{f} \in \mathcal{F}.$$

Types of function class $\mathcal{F}$:

1. parametric: $\mathcal{F} := \{f_\beta : \beta \in \mathcal{B}\}$, parameter β in a parameter set $\mathcal{B}$
2. nonparametric: $\mathcal{F}$ can be non explicitly described (*e.g.*, k-NN)

In both cases, $\mathcal{F}$ also depends on hyper-parameters γ, *i.e.*, $\mathcal{F} \equiv \mathcal{F}_\gamma$.

We assume a parametric model with parameters $\beta = (\beta_0, \cdots, \beta_p)^\top$.

For instance, f is linear in β and X:

$$f \in \mathcal{F} := \{f_\beta(X) := \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p : \beta \in \mathbb{R}^{p+1}\}. \quad (1)$$

$\Rightarrow$ learning (fitting) f (with $\mathcal{A}$)
$\equiv$ minimizing a cost function ℓ over $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^N$ (*e.g.*, the squared loss).

Remarks:

- Cost ℓ adjusted to (an estimate of) the prediction error (the *risk*, see later)
- p (with $p+1$ the dimension of β) is a hyperparameter, *i.e.*, $\{p\} \in \gamma$.

Extra information: We can generalize (1) as

$$f_\beta(X) := \beta^\top \varphi(X) = \beta_0 \varphi_0(X) + \cdots + \beta_{q-1} \varphi_{q-1}(X),$$

with $\varphi(X) = (\varphi_0(X), \ldots, \varphi_{q-1}(X))^\top \in \mathbb{R}^q$ a feature vector (& $q \geqslant p$).

For instance, $\varphi(X) \equiv$ monomials of X_i such as $X_0, X_0^2, X_0 X_1, X_1^2, \ldots$
sines/cosines of the X_i, wavelets ...

Examples of parametric models:

- linear model and least squares (see before)
- logistic regression (see later)
- (deep) neural networks, ... (not covered)

Pros:

- can be easy to optimize (*e.g.*, LS + numerical algebra);
- many algorithms available for this optimization (*e.g.*, gradient descent);
- applicable to any dimensions.

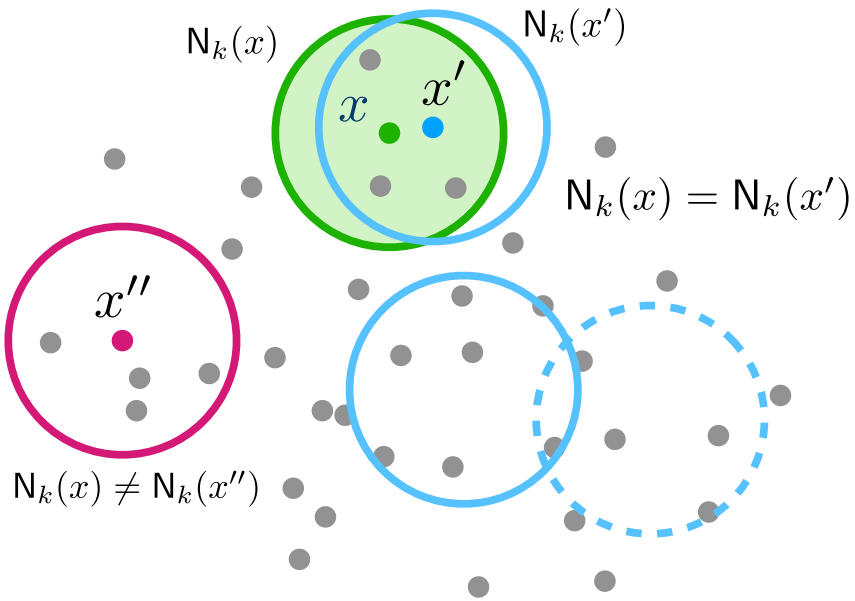
Cons:

- one must control β and p to avoid overfitting;
- one often needs a good feature vector φ for good performances;
- can be hard to optimize (*e.g.*, for neural networks);
- can be hard to interpret beyond linear models (*e.g.*, for neural networks).

Examples: local averaging, k-Nearest Neighbors (k-NN), thin-plate splines, ...

Principles:

- No explicit assumption on the functional form of f.
- But implicit assumption anyway:
  For instance, for k-NN, $\mathcal{F} \approx$ piecewise constant functions.



- ⚠ $\exists$ hyperparameters $\gamma \equiv$ not learned at training (*e.g.*, k in k-NN)
- Objective: find $\hat{f}$ as close as possible to $\mathcal{T}$ without to be too *wiggly*

Pros:

- They can fit a larger set of unknown functions f.
- Higher flexibility (or *smoothness*).

Cons:

- Need much more data (observations) at training to be accurate.
- Flexibility level must be found (*hyperparameter*) by model selection.
- Much less interpretable, hard to use for inference.

Learning a model in scikit-learn



1. *Selecting* a regressor/classifier, *i.e.*, a model, *e.g.*, via a constructor such as

Regressor example:

```
from sklearn import linear_model
...
reg = linear_model.LinearRegression()
```

Classifier example:

```
from sklearn import neighbors
...
kNN = neighbors.KNeighborsClassifier(
                    n_neighbors = 10)
```

2. *Fitting* ($\equiv$ *learning*) thanks to `reg.fit(X,y)` method (with $\mathcal{T} \equiv \{\texttt{X}, \texttt{y}\}$), with observations $\texttt{X} = \boldsymbol{X} \in \mathbb{R}^{N \times p}$ and labels $\texttt{y} = \boldsymbol{y} \in \mathbb{R}^{N}$, *e.g.*,

```
X = [[0, 0], [1, 1], [2, 2]]
y = [0, 1, 2]
reg.fit(X, y)
# reg.coef_ gives array([0.5, 0.5]) = beta
```

and similarly for the classifier.

(3. *Predicting*, using the learned model, with `reg.predict(x_new)`.)

More on this later + in the *practical sessions* and *hackathons*.

A **good model** $\hat{f}$ is such that on a new input x with label y,

we get small prediction error: $\hat{f}(x) \underset{\uparrow \text{error}}{\approx} y \quad \Leftrightarrow \quad$ low generalization error

$\Rightarrow$ theoretical context: statistical decision theory (SDT) (more details later)

Sources of error? (an analysis in expectation)

For regression, assume $\hat{f}$ and X *fixed* in $Y = f(X) + \varepsilon$ (with $\mathbb{E}\varepsilon = 0$),

$$\underbrace{\text{Risk of } \hat{f} \equiv \mathrm{R}(\hat{f})}_{\text{for the squared loss}} := \mathbb{E}\underbrace{(Y - \hat{f}(X))^2}_{\text{squared loss}} = \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\mathbb{V}(\varepsilon)}_{\text{Irreducible}}$$

- Reducible error: $\hat{f}$ is not a perfect estimate for f,
  but it can be improved (better model $\mathcal{F}$, better (hyper) parameters $\beta, \gamma, \ldots$)
- Irreducible error: even if $\hat{f}$ is perfect, still a non-zero error
  Why? Because of noise (ε), unmeasured variable ($\exists X_{p+1}$?)

Assessing model accuracy in practice:

Given observed data $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^N$, we can compute the mean squared error (MSE) or misclassification error rate (Err):

$$\text{MSE}(\mathcal{T}) := \frac{1}{N} \sum_{i=1}^N \left(y_i - \hat{f}(x_i)\right)^2, \qquad \text{(for regression)}$$

$$\text{Err}(\mathcal{T}) := \frac{1}{N} \sum_{i=1}^N I\left(y_i \neq \hat{f}(x_i)\right). \qquad \text{(for classification)}$$

Both are empirical risks (training errors), *i.e.*, obtained from

$$\hat{\text{R}}(\hat{f}) := \underbrace{\frac{1}{N} \sum_{i=1}^N}_{\approx \mathbb{E}} \ell(y_i, \hat{f}(x_i)) \underset{N \to \infty}{\longrightarrow} \text{R}(\hat{f}),$$

- MSE: $\ell(a, b) = (a - b)^2$ for $a, b \in \mathbb{R} \equiv$ "squared loss";
- Err: $\ell(a, b) = I(a \neq b)$ for $a, b \in \mathcal{G} \equiv$ "0/1 loss"

What is the prediction error on unseen test data?

- A priori $\hat{\mathrm{R}}(\hat{f})$ and the Risk $\mathrm{R}(\hat{f})$ are not related
- With enough *flexibility*, we can build $\hat{f}$ such that $\hat{\mathrm{R}}(\hat{f}) = 0$ on $\mathcal{T}$, but $\mathrm{R}(\hat{f}) \neq 0$, *i.e.*, we cannot be sure that on a new x_0, $y_0 \approx \hat{f}(x_0)$.

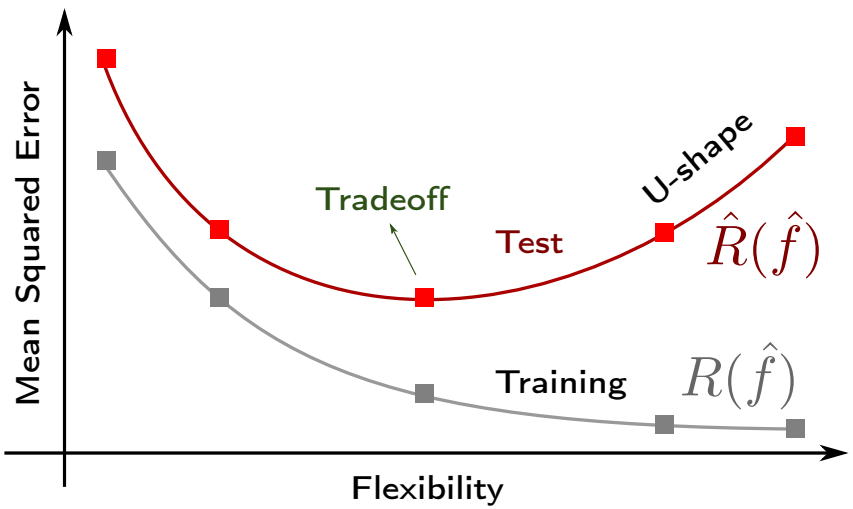
  $\Rightarrow$ We could start to overfit the dataset, start to learn errors/noise.

In fact, we can only control the reducible error.

As will be clear later, this

- depends on a tradeoff between competing forces, the variance and the bias,
- whose interaction depends on the model flexibility (aka complexity).

True Bias-Variance tradeoff only observable on test data.



What if no/not enough test data? $\rightarrow$ estimate test error on training data!

- *e.g.*, cross-validation (CV), K-fold CV, bootstrap, resampling techniques, ...